

Mobile App Programming Milestone 2

Team Members

Name	ID	Contribution	Screen Dev	Screen Review
Aljazi Alabdulqader	2240005506	UI\UX Wireframes Design	Closet, Sign-in	Jory Alnaim
Fajer Almusabbah	2240001917	UI\UX Wireframes Design	Tracker, Spread	Fajr Abzoah
Fajr Abuzoah	2240005562	UI\UX Wireframes Design	Profile, Tagging	Fajer Almusabbah
Raneem Almusallam	2240002822	Listing Jetpack Libraries	Home	Aljazi Alabdulqader
Jory Alnaim	2240007310	Navigation Map	Archive	Ethar Alujaiyan
Lujain Al Darweesh	2240004937	Specified Mystiq's functions for 7 Interfaces	Vault	Ethar Alujaiyan
Ethar Alujaiyan	2240007459	Specified Mystiq's functions for 3 Interfaces, and identified the data layer of the app.	Camera	Lujain Al Darweesh

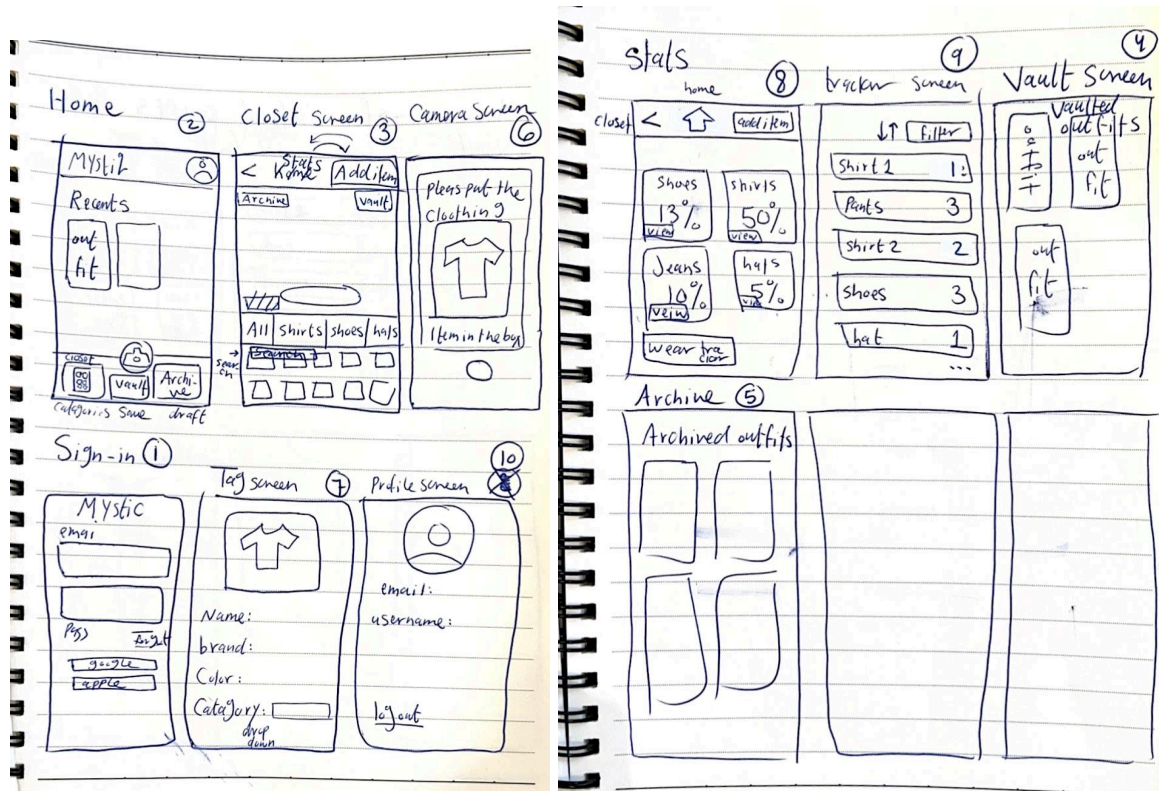
1. Wireframes

There will be 10 main Screens. We first Sketched the Screens on paper, then Digitized it to create a mockup.

1. Home Screen
2. Closet Screen
3. Camera Screen
4. Tagging Screen
5. Spread Screen
6. Vault Screen
7. Archive Screen
8. Sign-in Screen
9. Profile Screen

10. Log-out Screen

1.1 Screens Sketch



1.2 Screens Mockup



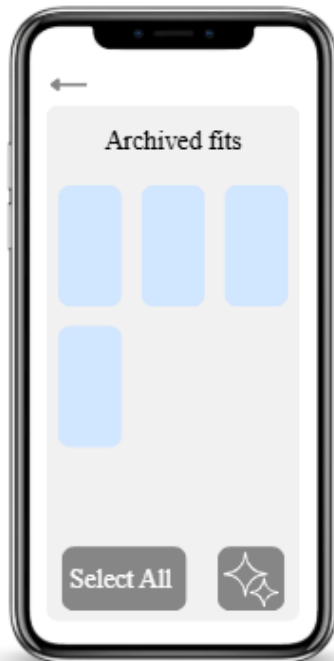
Sign-in



Home



Closet



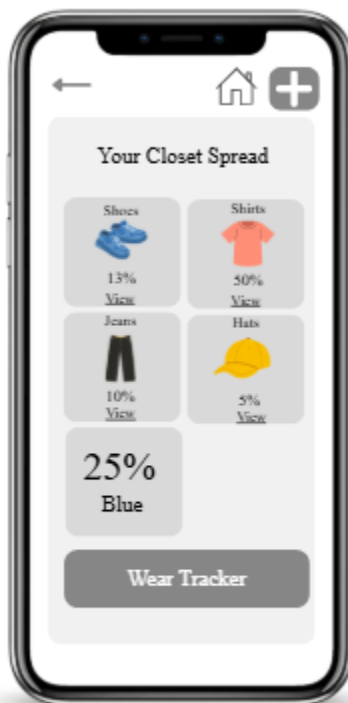
Archive



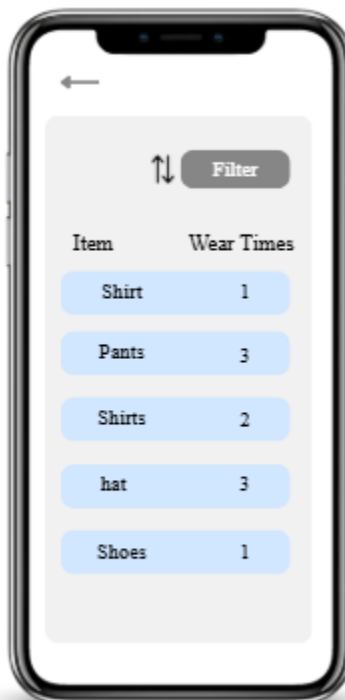
Camera



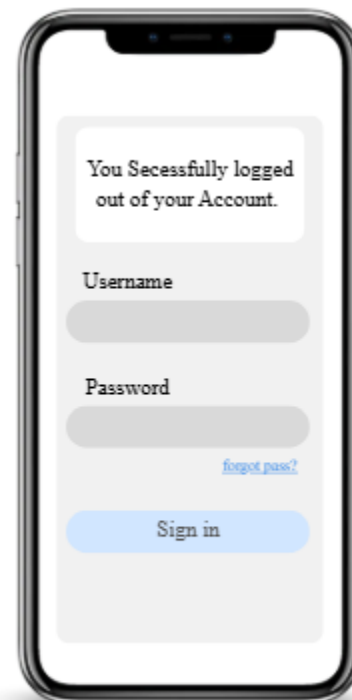
Tagging



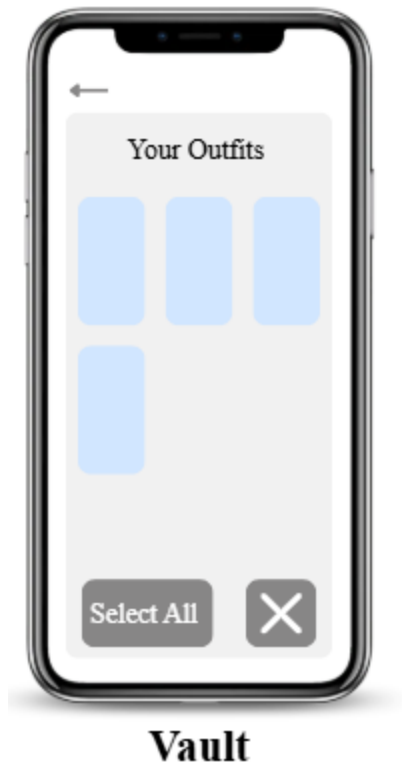
Spread



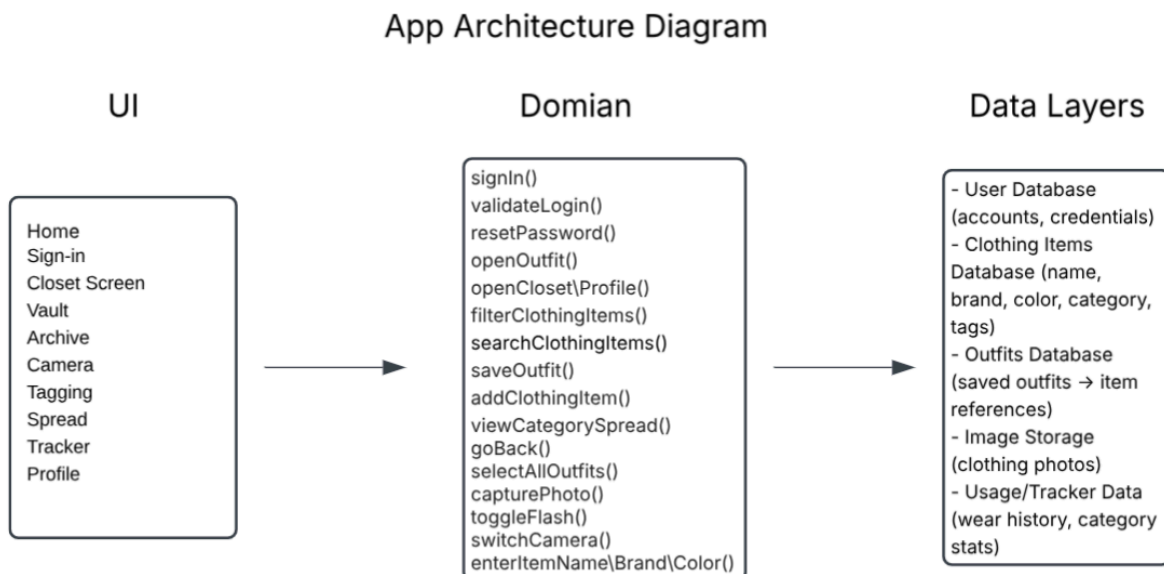
Tracker



Profile

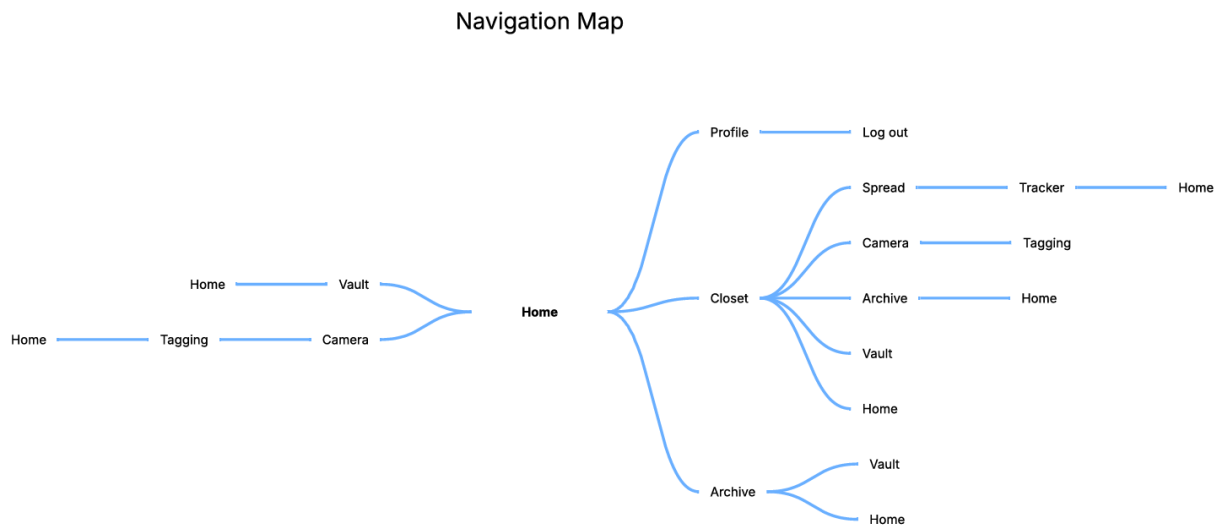


2. App Architecture Diagram



3. Navigation Map

The diagram below shows what screens you can traverse to from which screen.



4. Jetpack Libraries and Components

The table below shows the Libraries and Components that will be used in Mystiq.

Jetpack Library	Component	Functionality\Justification
CameraX	ProcessCameraProvider	Scanning: binds the camera to the screen lifecycle so it opens and closes automatically
CameraX	Preview	Scanning: shows the live camera feed while photographing a clothing item
CameraX	ImageCapture	Scanning: takes the photo and saves it as a file for the closet
CameraX	CameraXViewfinder (camera-compose)	Scanning: displays the camera preview

		inside Compose
Activity	<code>ActivityResultContracts.PickVisualMedia</code>	Scanning: lets the user upload an existing photo from the gallery
Activity	<code>RequestPermission</code>	Scanning: asks for camera permission at runtime
Room	<code>@Entity</code> Item	Scanning: stores each clothing item with its image path, name, and price
Room	<code>@Entity</code> Cabinet + <code>@ForeignKey</code>	Cabinets: creates custom categories (Shirts, Pants, Shoes) and links each item to one
Room	<code>@Relation</code> / <code>@Embedded</code>	Cabinets: loads a cabinet together with all of its items in one query
Room	<code>Flow</code> queries	Cabinets: the UI updates automatically when items are added, moved, or deleted
Compose Foundation	<code>LazyVerticalGrid</code>	Cabinets: shows the items of a cabinet as a scrollable photo grid
Compose Foundation	<code>LazyColumn</code>	Cabinets: shows the list of cabinets
Navigation Compose	<code>NavHost</code>	Cabinets: defines the screens (cabinet list, cabinet detail, preview space, and so on)
Navigation Compose	<code>NavController</code>	Cabinets: moves between screens, such

		as opening a cabinet
Compose Foundation	<code>Modifier.pointerInput + detectTransformGestures</code>	Preview Space: move, scale, and rotate items on the canvas
Compose Foundation	<code>Modifier.dragAndDropSource / dragAndDropTarget</code>	Preview Space: drag an item from the closet strip onto the canvas
Compose Foundation	<code>Modifier.graphicsLayer / rememberGraphicsLayer</code>	Preview Space: applies position, scale, and rotation, and captures the canvas as a look thumbnail
Lifecycle	<code>ViewModel</code>	Preview Space: holds the canvas state so it survives rotation
Lifecycle	<code>SavedStateHandle</code>	Preview Space: restores the canvas after process death
Room	<code>@Entity</code> Outfit with <code>isDraft</code> flag	Preview Space: saves drafts temporarily, with each item's x, y, scale, and rotation
DataStore	<code>Preferences DataStore</code>	Preview Space: optional lightweight alternative for storing a single draft
Room	<code>OutfitItemCrossRef + @Junction</code>	Outfit Builder: many-to-many link between outfits and items (top, bottom, shoes, accessories)
Compose Foundation	<code>HorizontalPager</code>	Outfit Builder: swipe through tops, bottoms, and shoes to pick each slot

Material 3	<code>Scaffold</code>	Outfit Builder: the screen structure (top bar, content, save button)
Material 3	<code>Card</code>	Outfit Builder: shows each item as a selectable card
Material 3	<code>FilledTonalButton</code>	Outfit Builder: the "Save look" action
Material 3	<code>ModalBottomSheet</code>	Outfit Builder: the accessories picker
Room	<code>@Fts4</code> entity	Search: full-text search over item names and tags
Room	<code>Tag</code> entity + cross-ref	Tagging: attach multiple tags to an item (for example where it was bought)
Material 3	<code>SearchBar / OutlinedTextField</code>	Search: the search input and the item name field
Material 3	<code>FilterChip / InputChip</code>	Tagging: filter by tag and add or remove tags on an item
Compose Foundation	<code>FlowRow</code>	Tagging: wraps tag chips onto multiple lines
Lifecycle	<code>viewModelScope + collectAsStateWithLifecycle</code>	Search: collects live search results (use Flow <code>debounce</code> and <code>flatMapLatest</code> for typing)
Room	Aggregate queries (<code>COUNT</code> , <code>GROUP BY</code>)	Closet Stats: percentage of shoes versus tops and similar breakdowns

Palette (<code>androidx.palette</code>)	<code>Palette.from(bitmap)</code>	Closet Stats: extracts the dominant color of each item for most-worn colors
Compose Foundation	<code>Canvas (drawArc, drawRect)</code>	Closet Stats: draws donut and bar charts
Room	<code>@Entity</code> WearLog (itemId, date)	Wear Tracker: records each time an item is worn
Room	<code>wearCount</code> query	Wear Tracker: finds underused items and calculates cost-per-wear (price ÷ wears)
WorkManager	<code>PeriodicWorkRequest</code>	Wear Tracker: schedules a daily "what did you wear?" reminder
WorkManager	<code>CoroutineWorker</code>	Wear Tracker: runs the reminder work in the background
Core	<code>NotificationCompat</code>	Wear Tracker: builds the reminder notification
Core	<code>NotificationManagerCompat</code>	Wear Tracker: shows the notification (needs the notification permission on Android 13+)
Material 3	<code>DatePicker</code>	Wear Tracker: log a wear for a past date
Material 3	<code>Checkbox / Switch</code>	Wear Tracker: quickly tick the items worn today
Compose BOM	<code>ui, foundation, material3,</code>	All features: one BOM keeps all Compose versions

	<code>ui-tooling-preview</code>	consistent
Lifecycle	<code>lifecycle-viewmodel-compose</code>	All features: <code>viewModel()</code> inside composables
Hilt (<code>androidx.hilt</code>)	<code>hilt-navigation-compose</code> <code>(hiltViewModel())</code>	All features: injects the database and repositories into ViewModels
Core	<code>core-ktx</code>	All features: Kotlin extension functions for Android APIs
Activity	<code>activity-compose</code> <code>(setContent)</code>	All features: hosts the Compose UI in <code>MainActivity</code>